Glynac Enterprise HashiCorp Stack

Production Architecture, Deployment & Automation Deep Dive

Submitted By:
Soham Mukherjee

Table of Contents

Table of Contents 2

1. Executive Summary 4

Key Objectives and SLOs (Service Level Objectives): . . . 4

2. Overview of Glynac's HashiCorp Stack 5

2.1 High-Level Architecture . . . 5

2.2 Control Plane vs Data Plane Design . . . 5

3. Nomad: Workload Orchestration 6

3.1 Installation & Service Creation . . . 6

3.2 Nomad Server Configuration (HCL) . . . 6

3.3 Example Job File with Consul Connect (Service Mesh) . . . 6

3.4 Rolling Upgrades & Rollbacks . . . 7

4. Consul: Service Discovery & Mesh 8

4.1 Server Configuration (JSON) . . . 8

4.2 ACL Bootstrap & Intentions . . . 8

5. Vault: Secrets Management 9

5.1 Bootstrapping Flow . . . 9

5.2 Vault Server Configuration (HCL) . . . 10

5.3 Initialization and Auth Methods . . . 11

5.4 Vault Policy Definition (HCL) . . . 11

6. Bootstrapping & Provisioning 12

6.1 systemd Unit File (Nomad Example) . . . 12

6.2 Cloud-Init User-Data Snippet . . . 12

7. Backups, DR & Recovery Playbooks 13

7.1 Snapshot Commands . . . 13

8. CI/CD and GitOps Integration 14

8.1 GitHub Actions Workflow Snippet . . . 14

8.2 Safe Deploy Bash Wrapper . . . 14

9. Observability & Alerting **16**

9.1 Prometheus Scrape Configuration 16

10. Best Practices & Operational Runbooks **17**

Node Failure Runbook (Nomad/Consul) 17

Consul Leader Election Troubleshooting 17

Smoke Tests 17

Appendix: Generated Artifacts **18**

1. Executive Summary

This document outlines the architectural blueprint, operational runbooks, and programmatic configurations for Glynac's enterprise-grade HashiCorp stack. Designed to achieve high availability (HA), robust security via Zero Trust principles, and massive scalability, the infrastructure relies on Nomad for workload orchestration, Consul for service discovery and mesh networking, and Vault for identity-based secrets management.

By standardizing on the HashiCorp cloud operating model, Glynac decouples its application delivery from underlying proprietary cloud implementations. This ensures multi-cloud portability, automated disaster recovery, and mathematically verifiable cryptographic security postures across all operational environments.

Key Objectives and SLOs (Service Level Objectives):

- **Control Plane Availability:** 99.99% uptime for Nomad, Consul, and Vault clusters.
- **Deployment Velocity:** Reduce CI/CD deployment times to under 3 minutes globally.
- **Security Posture:** 100% of internal microservice traffic encrypted via mTLS (Consul Connect).
- **Secrets Management:** Zero hardcoded credentials; 100% reliance on dynamic, short-lived Vault tokens.
- **Disaster Recovery:** RTO (Recovery Time Objective) of < 15 minutes; RPO (Recovery Point Objective) of < 5 minutes via Raft snapshots.

2. Overview of Glynac's HashiCorp Stack

Glynac's infrastructure handles high-throughput financial transactions and real-time data streaming. Traditional Kubernetes clusters introduced massive operational overhead and complexity that hindered developer velocity. The pivot to Nomad, Consul, and Vault provided a single-binary approach to operations, drastically reducing the cognitive load on Site Reliability Engineers (SREs).

2.1 High-Level Architecture

The architecture spans multiple regions to ensure fault tolerance. The primary datacenter (`us-east-1`) houses the active control planes, while the secondary (`us-west-2`) acts as a warm standby with WAN federation enabled.

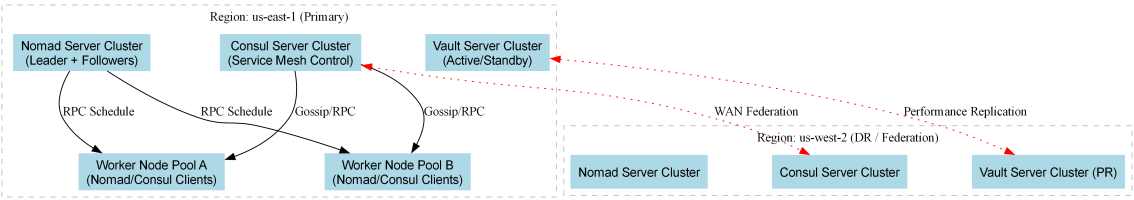

Figure 1: *High-Level Architecture demonstrating Multi-Region Nomad, Consul, and Vault clusters.*

2.2 Control Plane vs Data Plane Design

Separation of concerns is critical. The Control Plane consists of dedicated, highly-provisioned EC2 instances running server agents. These nodes do NOT run application workloads. They handle Raft consensus, leader election, and cryptographic signing.

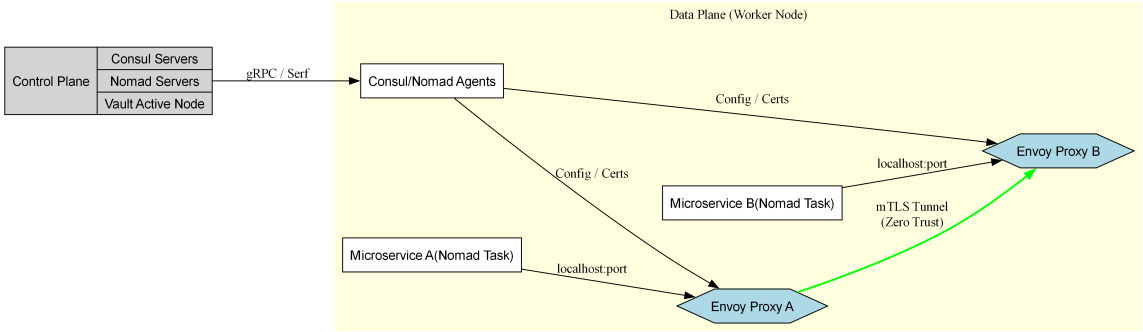

Figure 2: *Control Plane vs Data Plane illustrating Envoy proxy sidecars and mTLS tunnels.*

3. Nomad: Workload Orchestration

Nomad is configured to schedule both Docker containers and raw Java/Go binaries (Exec driver). The server cluster consists of 5 nodes to sustain up to 2 simultaneous node failures without losing quorum.

3.1 Installation & Service Creation

```
# Install Nomad via HashiCorp APT repo
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo apt-key add -
sudo apt-add-repository "deb [arch=amd64] https://apt.releases.hashicorp.com $(lsb_release -cs) main"
sudo apt-get update && sudo apt-get install nomad -y

# Verify installation
nomad --version
```

3.2 Nomad Server Configuration (HCL)

```
data_dir  = "/opt/nomad/data"
bind_addr = "0.0.0.0"

server {
  enabled          = true
  bootstrap_expect = 5

  server_join {
    retry_join = ["provider=aws tag_key=Role tag_value=nomad-server"]
  }
}

acl {
  enabled = true
}

tls {
  http = true
  rpc  = true
  ca_file   = "/opt/nomad/tls/nomad-ca.pem"
  cert_file = "/opt/nomad/tls/server.pem"
  key_file  = "/opt/nomad/tls/server-key.pem"
}
```

3.3 Example Job File with Consul Connect (Service Mesh)

```
job "payment-gateway" {
  datacenters = ["us-east-1"]
  type        = "service"

  group "api" {
    count = 3

    network {
      mode = "bridge"
    }
```

```
    service {
      name = "payment-api"
      port = "9090"

      connect {
        sidecar_service {}
      }

      check {
        type     = "http"
        path     = "/health"
        interval = "10s"
        timeout  = "2s"
      }
    }

    task "gateway" {
      driver = "docker"

      vault {
        policies = ["payment-db-read"]
      }

      config {
        image = "glynac/payment-gateway:v1.4.2"
      }

      template {
        data = <<EOH
          DB_USER="{{ with secret "database/creds/payment" }}{{ .Data.username }}{{ end }}"
          DB_PASS="{{ with secret "database/creds/payment" }}{{ .Data.password }}{{ end }}"
        EOH
        destination = "secrets/db.env"
        env         = true
      }
    }
  }
}
```

3.4 Rolling Upgrades & Rollbacks

To deploy the job safely, use the deployment commands:

```
nomad job run payment-gateway.nomad
nomad deployment status <deployment_id>
nomad deployment promote <deployment_id>
nomad deployment fail <deployment_id> # Triggers automatic rollback
```

4. Consul: Service Discovery & Mesh

Consul tracks the real-time IP and port of every ephemeral container. Consul Connect injects Envoy sidecars to establish mutual TLS (mTLS) between microservices.

4.1 Server Configuration (JSON)

```
{
  "datacenter": "us-east-1",
  "data_dir": "/opt/consul",
  "server": true,
  "bootstrap_expect": 5,
  "ui_config": {
    "enabled": true
  },
  "retry_join": ["provider=aws tag_key=Role tag_value=consul-server"],
  "encrypt": "<GOSSIP_ENCRYPTION_KEY>",
  "verify_incoming": true,
  "verify_outgoing": true,
  "verify_server_hostname": true,
  "ca_file": "/opt/consul/tls/consul-ca.pem",
  "cert_file": "/opt/consul/tls/server.pem",
  "key_file": "/opt/consul/tls/server-key.pem",
  "acl": {
    "enabled": true,
    "default_policy": "deny",
    "enable_token_persistence": true
  },
  "connect": {
    "enabled": true
  }
}
```

4.2 ACL Bootstrap & Intentions

```
# Bootstrap the ACL system (Run ONCE)
consul acl bootstrap

# Create an Intention to allow Web to talk to API via mTLS
consul intention create -allow web payment-api

# Block all other traffic implicitly (Zero Trust)
consul intention create -deny * *
```

5. Vault: Secrets Management

Vault is the absolute source of truth for cryptographic material. Glynac utilizes AWS KMS for Auto-Unseal, meaning operators do not need to manually enter Shamir key shares during a server reboot.

5.1 Bootstrapping Flow

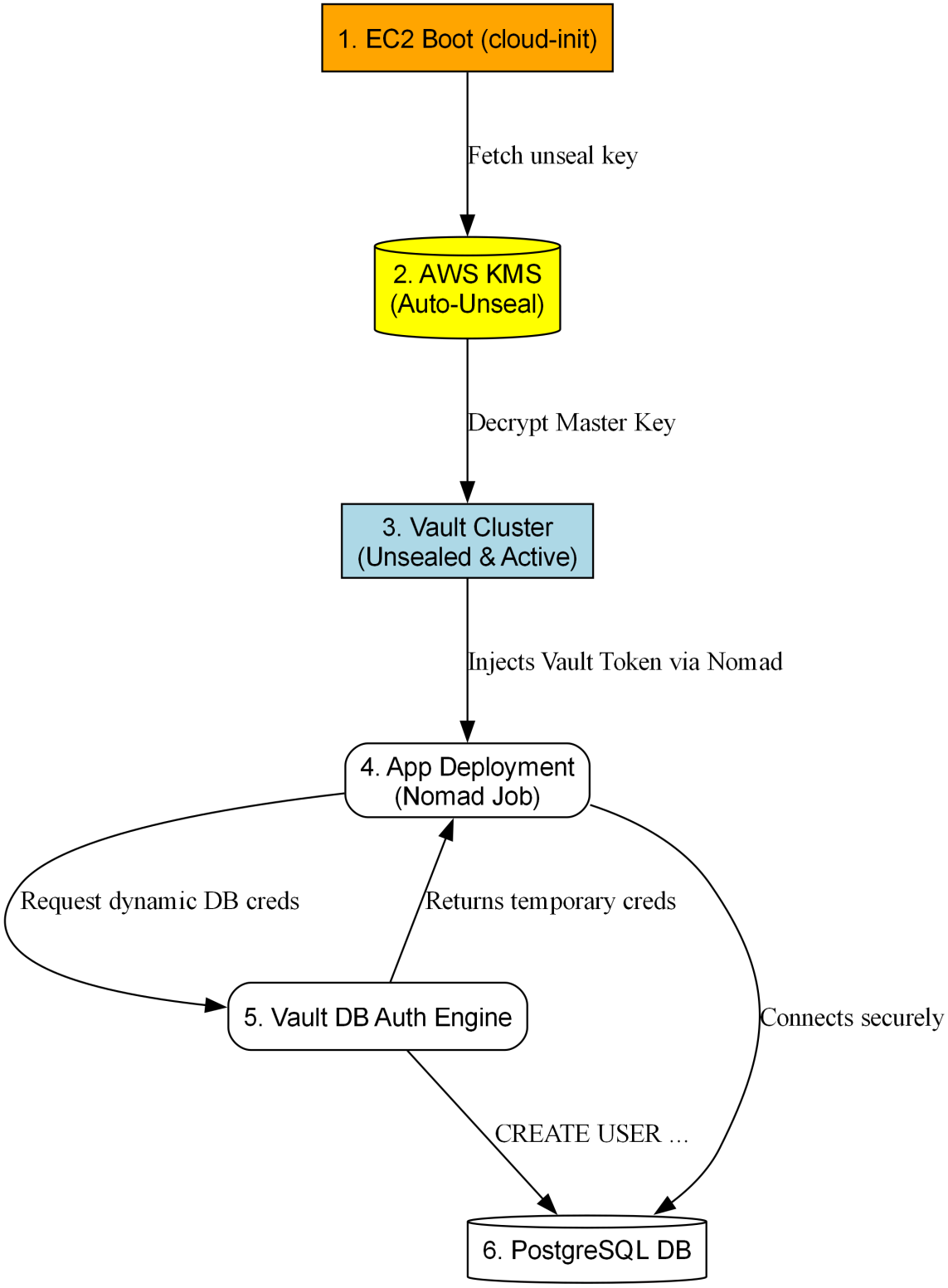

Figure 3: *Vault auto-unseal and dynamic credential issuance flow.*

5.2 Vault Server Configuration (HCL)

```
storage "raft" {
  path    = "/opt/vault/data"
  node_id = "vault-server-01"
}

listener "tcp" {
  address       = "0.0.0.0:8200"
  tls_cert_file = "/opt/vault/tls/vault.crt"
  tls_key_file  = "/opt/vault/tls/vault.key"
}

seal "awskms" {
  region     = "us-east-1"
  kms_key_id = "<AWS_KMS_KEY_ID>"
}

api_addr = "https://vault.glynac.internal:8200"
cluster_addr = "https://vault-server-01:8201"
ui = true
```

5.3 Initialization and Auth Methods

```
# Initialize Vault (Generates recovery keys, not unseal keys due to KMS)
vault operator init

# Enable AWS Auth for EC2/Nomad clients
vault auth enable aws
vault write auth/aws/config/client secret_key=<SECRET> access_key=<ACCESS>

# Configure Database Secrets Engine
vault secrets enable database
vault write database/config/postgresql \
    plugin_name=postgresql-database-plugin \
    connection_url="postgresql://{{username}}:{{password}}@db.glynac.internal:5432/postgres" \
    allowed_roles="payment-role"

# Create a Dynamic Role (1-hour TTL)
vault write database/roles/payment-role \
    db_name=postgresql \
    creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL '{{ex
    default_ttl="1h" \
    max_ttl="24h"
```

5.4 Vault Policy Definition (HCL)

```
# payment-policy.hcl
path "database/creds/payment-role" {
  capabilities = ["read"]
}
path "secret/data/payment/*" {
  capabilities = ["read", "list"]
}
```

6. Bootstrapping & Provisioning

Instances are provisioned immutably via Terraform. When an EC2 instance launches, `cloud-init` configures the OS, drops the certificates, and starts the systemd services.

6.1 systemd Unit File (Nomad Example)

```
[Unit]
Description=Nomad
Documentation=https://nomadproject.io/docs/
Wants=network-online.target
After=network-online.target

[Service]
ExecReload=/bin/kill -HUP $MAINPID
ExecStart=/usr/local/bin/nomad agent -config /etc/nomad.d
KillMode=process
KillSignal=SIGINT
LimitNOFILE=infinity
LimitNPROC=infinity
Restart=on-failure
RestartSec=2
StartLimitBurst=3
StartLimitIntervalSec=10
TasksMax=infinity

[Install]
WantedBy=multi-user.target
```

6.2 Cloud-Init User-Data Snippet

```
#!/bin/bash
set -e

echo "Configuring Consul Agent..."
mkdir -p /etc/consul.d /opt/consul
chown -R consul:consul /opt/consul

systemctl enable consul
systemctl start consul

echo "Configuring Nomad Agent..."
systemctl enable nomad
systemctl start nomad

echo "Node Bootstrapping Complete."
```

7. Backups, DR & Recovery Playbooks

State loss in Consul or Vault results in total infrastructure outage. Automated Raft snapshots are mandatory.

7.1 Snapshot Commands

```
# Consul Snapshot
consul snapshot save /mnt/efs/backups/consul/backup_$(date +%F).snap

# Vault Snapshot
vault operator raft snapshot save /mnt/efs/backups/vault/vault_$(date +%F).snap

# Restore Playbook (In event of catastrophic loss)
systemctl stop vault
vault operator raft snapshot restore /mnt/efs/backups/vault/vault_latest.snap
systemctl start vault
```

WARNING: Restoring a Vault snapshot rewrites the entire cryptographic state. All active dynamic tokens and leases issued AFTER the snapshot was taken will immediately be invalidated.

8. CI/CD and GitOps Integration

Infrastructure deployments are executed entirely via GitHub Actions runners authenticated via OIDC into Vault to retrieve AWS credentials and Nomad tokens.

8.1 GitHub Actions Workflow Snippet

```
name: Deploy Payment Gateway
on:
  push:
    branches: [ "main" ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Import Secrets from Vault
        uses: hashicorp/vault-action@v2
        with:
          url: https://vault.glynac.internal:8200
          method: jwt
          role: github-actions
          secrets: |
            secret/data/ci/nomad token | NOMAD_TOKEN ;

      - name: Setup Nomad
        uses: hashicorp/setup-nomad@v2

      - name: Validate Nomad Job
        run: nomad job plan payment-gateway.nomad
        env:
          NOMAD_ADDR: https://nomad.glynac.internal:4646

      - name: Run Nomad Job
        run: nomad job run payment-gateway.nomad
        env:
          NOMAD_ADDR: https://nomad.glynac.internal:4646
```

8.2 Safe Deploy Bash Wrapper

```
#!/bin/bash
set -eo pipefail

echo "Running syntax validation..."
nomad job validate $1

echo "Executing Plan..."
nomad job plan $1 || true # Plan returns exit code 1 if there are changes

echo "Applying configuration to cluster..."
nomad job run $1
```

```
echo "Waiting for health checks..."
sleep 15
nomad job status $(grep 'name' $1 | awk '{print $3}' | tr -d '"')
```

9. Observability & Alerting

All HashiCorp tools expose a `/v1/metrics` endpoint natively formatted for Prometheus. Telemetry is collected and visualized in Grafana.

9.1 Prometheus Scrape Configuration

```
scrape_configs:
  - job_name: 'consul'
    consul_sd_configs:
      - server: 'localhost:8500'
        services: ['consul']
    metrics_path: /v1/agent/metrics
    params:
      format: ['prometheus']

  - job_name: 'nomad'
    consul_sd_configs:
      - server: 'localhost:8500'
        services: ['nomad-client', 'nomad']
    metrics_path: /v1/metrics
    params:
      format: ['prometheus']

  - job_name: 'vault'
    metrics_path: /v1/sys/metrics
    scheme: https
    tls_config:
      insecure_skip_verify: true
    bearer_token: <PROMETHEUS_VAULT_TOKEN>
    static_configs:
      - targets: ['vault.glynac.internal:8200']
```

Critical Alerts to Configure:

- **Vault Sealed Status:** Alert SRE immediately if `vault.core.unsealed` == 0.
- **Raft Leadership Transitions:** Spikes in `consul.raft.leader.lastContact` indicate network partitions.
- **Nomad Allocation Failures:** Alert on high rates of `nomad.client.allocations.failed`.

10. Best Practices & Operational Runbooks

Node Failure Runbook (Nomad/Consul)

If a physical worker node dies, Nomad will detect the loss of heartbeat within 2 minutes and automatically reschedule the tasks to healthy nodes. Operator intervention is generally not required unless a Control Plane node fails.

```
# Verify dead node status
consul members | grep failed
nomad node status

# Force remove dead node from Consul catalog
consul force-leave <node-name>

# Mark Nomad node as ineligible and drain (if partially alive)
nomad node drain -enable <node-id>
```

Consul Leader Election Troubleshooting

```
# Check Raft peers
consul operator raft list-peers

# If quorum is completely lost, create peers.json for manual recovery
echo '[
  {"id": "node1", "address": "10.0.0.1:8300", "non_voter": false},
  {"id": "node2", "address": "10.0.0.2:8300", "non_voter": false}
]' > /opt/consul/raft/peers.json
```

Smoke Tests

```
#!/bin/bash
# Validate Consul DNS
dig @127.0.0.1 -p 8600 payment-api.service.consul

# Validate Vault dynamic creds
vault read database/creds/payment-role

# Validate Nomad job health
nomad job status payment-gateway | grep "Status"
```

Appendix: Generated Artifacts

The following artifacts were programmatically generated alongside this document:

- high_level_arch.png
- control_data_plane.png
- bootstrapping_flow.png
- Glynac_HashiCorp_Stack_Soham_Mukherjee.pdf
- manifest.json